

COMPSYS704 2017 LAB3

Developing a SystemJ Program for Modelling a Simple Intelligent Machine – Workpiece Loader

14/07/2023

1. Introduction

The purpose of this lab is to introduce a methodology of modelling an intelligent machine using SystemJ language. The machine is intelligent as it consists of number of simple mechatronic components that can be actuated, their current statuses sensed and, by using a higher level software Controller, customised and made intelligent to allow interfacing with the other similar machines creating bigger industrial systems. The machine, in our case called Workpiece Loader, has the task of loading workpieces from a magazine barrel, using pneumatic-driven actuators, to a destination location where the workpiece joins specific manufacturing process. Destination location can be a static fixed location or dynamic fixed location (in the latter case it is implemented with a moving Conveyor). In this lab we will assume that the location is static. That means the provided workpiece can be moved further to join the manufacturing process only by using another machine that has capability to grip the workpiece and move it to the desired place in a manufacturing system.

We will use a model-driven approach to model both (1) Machine, with its sensors and actuators, and (2) its Controller, which generates proper control signals and brings the machine and its mechatronic components into the desired states. Sometimes, the Machine is called Plant and its model is called Machine Model or Plant Model.

Our goal is to develop models of the Machine and the Controller. These models are then integrated into the model of intelligent machine (IM) and can simulate machine operations. Simulation model will also include a visualisation of the machine operation, where only major states in which machine can be captured and have to reflect the state of the real Machine. Once the simulated model of the machine is validated, the developed Controller should be able to control the real Workpiece Loader directly, without modifications.

All individual parts, machine model, controller and visualisation, will be developed in a combination of the SystemJ and Java languages. Although we aim to make an intelligent machine, with functionality that can be easily extended as you will see during the course, the whole process is a completely software-only exercise.

2. Developing the simulation model

In this lab, you are going to develop control software, the Controller, for a Workpiece Loader (See Figure 1). The Workpiece Loader consists of three major mechatronic components:

1. *A magazine barrel*, which is independently loaded (manually or by another manipulator) and can hold up to 5 workpieces. The magazine has a photoeye sensor that can generate an indication when empty.
2. *An ejecting cylinder (pusher)*, which needs to retract to its initial state in order to be able to push a workpiece out of the magazine to an intermediate position from where the workpiece becomes available to be taken by

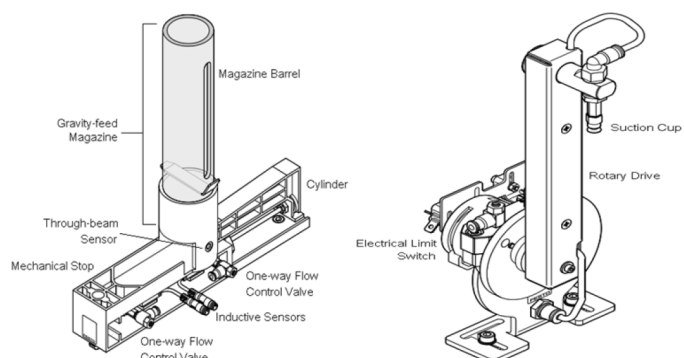
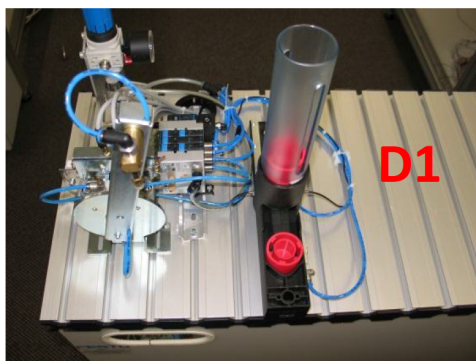


Figure 1. Workpiece Loader station

the rotary drive arm. The pusher generates feedback signals for the Controller when it reaches stable positions (fully retracted or fully extended).

3. A *rotary drive arm* is equipped with a suction cup that enables transfer of a workpiece from the position where it was left by the pusher to a pick up (destination) location. The arm has two stable positions/states that indicate (1) the arm is at the source position (retracted), where it finds a workpiece for transfer, and (2) the arm is at the destination position (extended) where the workpiece is handed over to another intelligent machine. The arm provides signals that indicate its current position. Also, added to the top of the arm is a suction cup which allows grabbing a workpiece at the source location and releasing it at the destination location.

This Workpiece Loader can be controlled via sets of actuators and sensors which are accessed and interfaced with a SystemJ program via input and output signals. As SystemJ will be used to implement the Controller, a control algorithm has to be proposed that enables synchronisation of operation of individual components into the desired operation. For example, simply specified, the goal is to bring the components into their initial positions and start a sequence of operations (such as pushing a workpiece from the magazine to the arm source position and moving the workpiece from the arm source position to the destination position). Operations have to be performed in a synchronised and safe way. In order to be able to connect your Workpiece Loader with other intelligent machines, it has to provide an interface such as being able to accept a command to deliver the next workpiece to a requestor and indicate that the workpiece is available at the destination location.

In this case the operation of the Controller, implemented as the SystemJ program, depends on its interactions with the Workpiece Loader components. Therefore, in order to test your program, you will also need a model of the machine and its parts, which provide input stimuli to and receives control signals that change the state of individual components from the Controller.

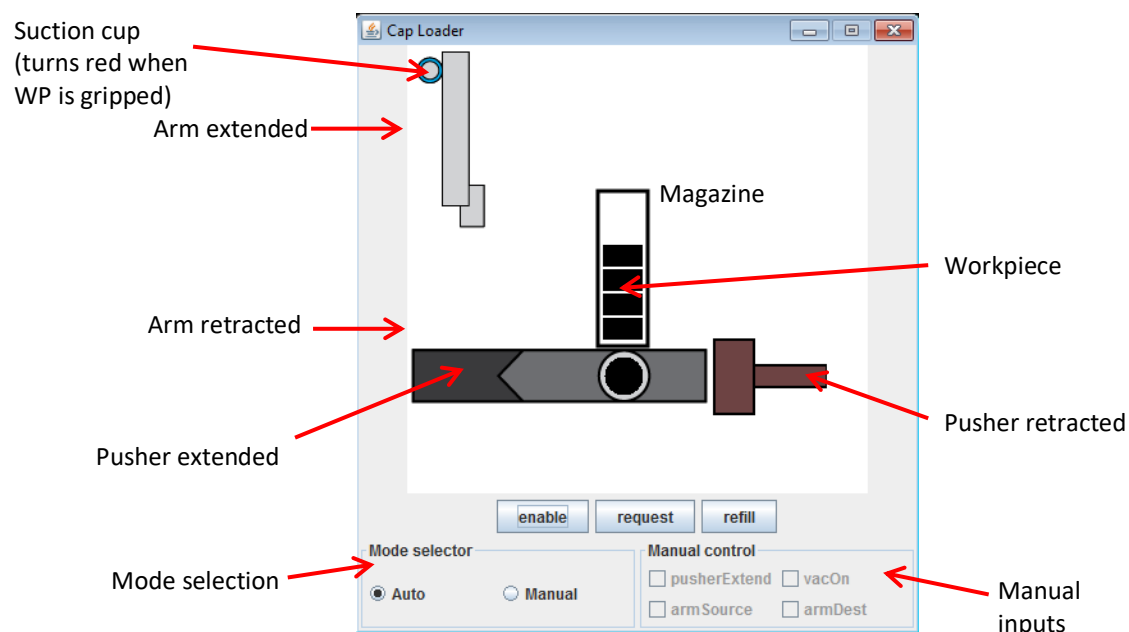
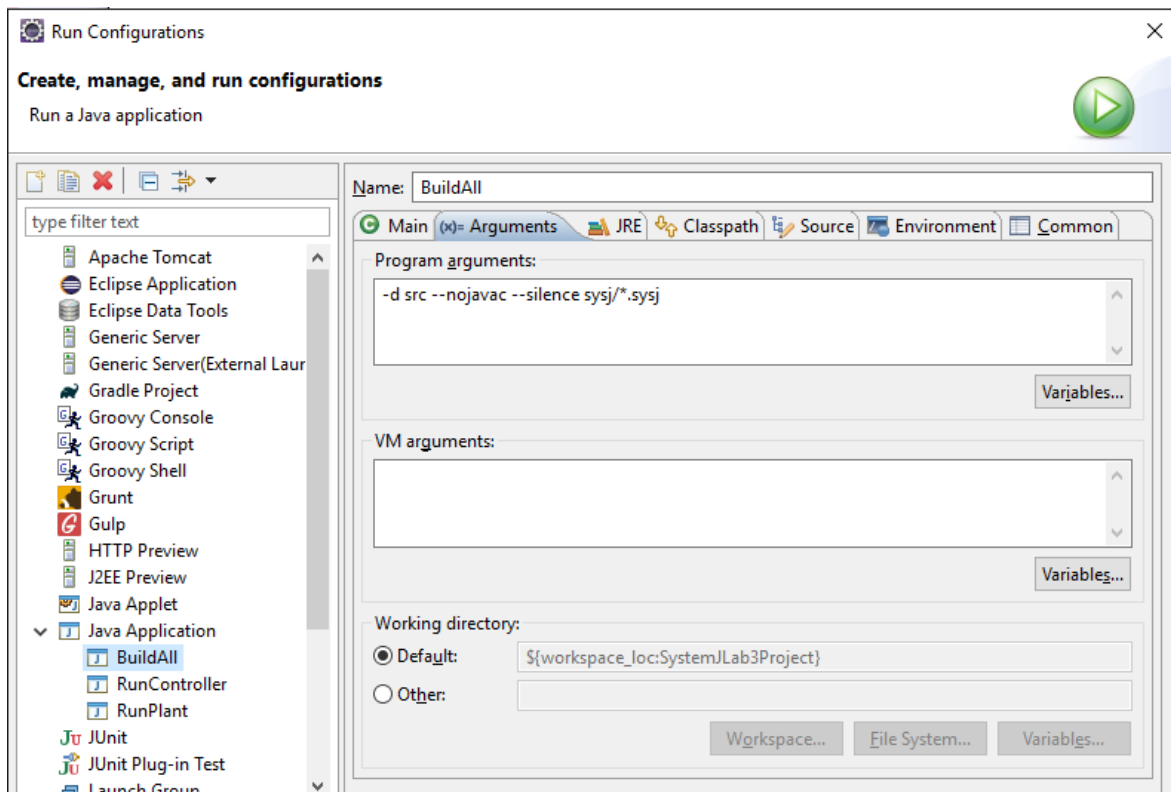
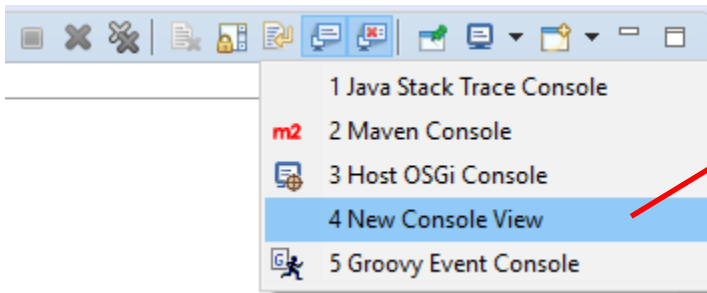


Figure 2. The Workpiece Loader simulation and visualisation environment

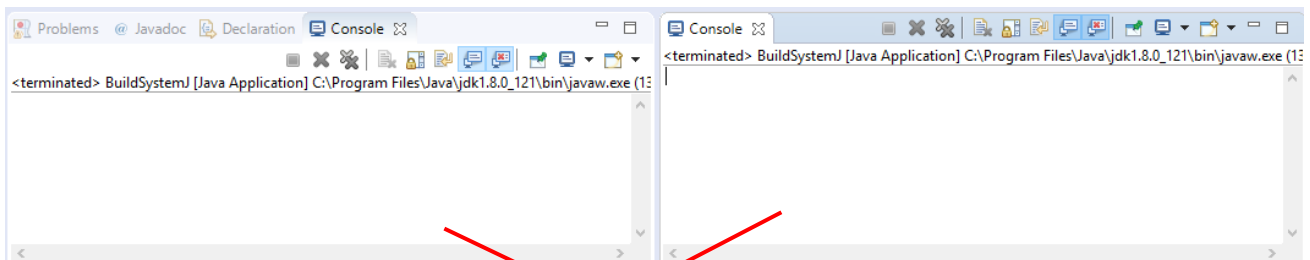
In this lab, you are already given a SystemJ model of the Workpiece Loader. Extract **SystemJLab3Project.zip** and import the extracted project in Eclipse. To compile the controller and the plant at once, select **Run > Run Configurations...** on the Eclipse menu bar, and run Java Application called **"BuildAll"** as shown below.



To run the controller and the plant, you first need to refresh the project by clicking the **SystemJLab3Project** in the project explorer window and press “F5”, then run configurations “**RunController**” and “**RunPlant**” (see the above picture) in sequence. You will see the plant clock-domain starts the visualization GUI during initialization (see plant.sysj). Each SystemJ program will create a new console where outputs are printed when needed. You can create a new console window as shown below so you can observe outputs from two SystemJ programs at the same time. Drag the newly created console window tab to a new position such that it splits into two windows.



Click to create a new console



Drag a console tab to split the window into two console views

3. I/O characteristics of the Workpiece Loader

Figure 3 shows an overview of the overall system. The SystemJ controller has a set of input and output signals to interface with the Workpiece Loader:

Output signals

- **pusherExtend**: continuous emission of this signal causes the pusher to extend. Absence of this signal will cause the pusher to retract.
- **vacOn**: continuous emission of this signal will turn the vacuum on.
- **armSource**: continuous emission of this signal causes the arm swivels to the source position to pick up a workpiece
- **armDest**: continuous emission of this signal causes the arm swivels to the destination position to drop off a workpiece

Input signals

- **pusherRetracted**: indicates the pusher has been retracted.
- **pusherExtended**: indicates the pusher has been extended.
- **WPgripped**: indicates the workpiece has been picked up by the suction cup.
- **armAtSource**: indicates the rotary drive is reached to the source to pick up a workpiece
- **armAtDest**: indicates the rotary drive is reached to the destination
- **empty**: indicates the magazine barrel is empty

Additionally the controller has an input signal “**request**”, coming from an external source, which initiates the loader operation:

- **request**: indicates a request for a new workpiece

The Workpiece Loader has input and output signals which match with the output and input signals of the Controller, respectively. Output signals of the Workpiece Loader are also forwarded to the Java UI to visualize current state of the Workpiece Loader.

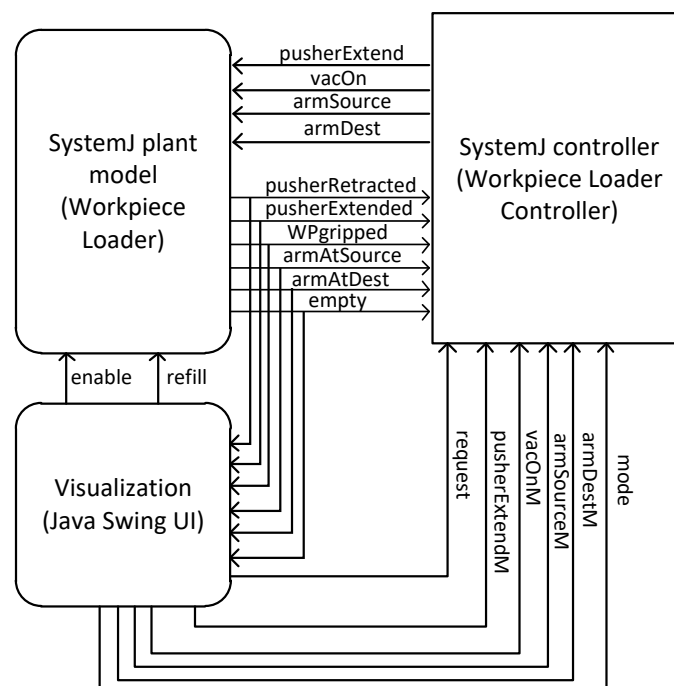


Figure 3. An overview of the system

Movement of the arm or the pusher takes certain time to reach their final position. However, in SystemJ the only notion of time is the logical tick. Real-time can be captured in SystemJ program using an external timer or a method described in [1]. In this lab, we will simplify the scenario so that movement of the components is triggered by a user via UI. For example, the arm or the pusher can only move when a user presses an “enable” button (see Figure 2). The enable button is provided as an external input that controls the operation of the plant during simulation and enables the transfer of the plant from one state to the next state. This button is to aid the functional validation of the Controller functionality. “request” and “refill” buttons are used to emulate a request for new workpieces from the Workpiece Loader and to refill the magazine when empty, respectively.

The SystemJ Controller also has an input signal from the UI, which can replace automatic operation of the Controller and bring the Controller into Manual mode:

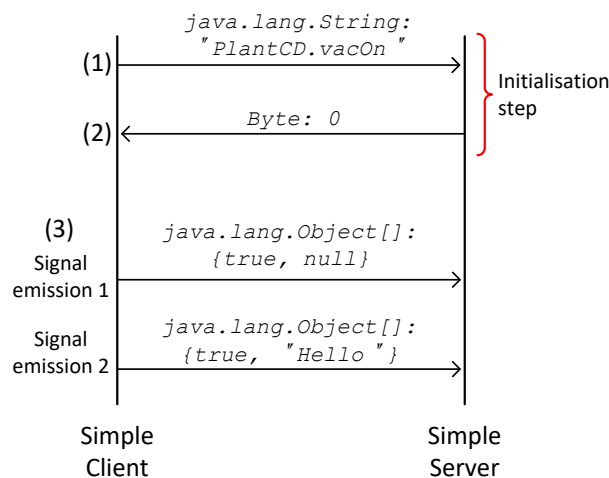
- **mode** : An integer value of this signal indicates an operation mode of the controller: “0” – automatic; “1” - manual.

When the controller is in manual mode, a user should be able to control output signals of the Controller via the Manual control panel shown in Figure 2. For example, if a checkbox ‘vacOn’ is ticked, the UI sets signal “vacOnM” to high, which is an input signal to the Controller. Consequently, the Controller should set output signal “vacOn” to high. Note that movements of the pusher and the arm are still triggered by the enable button in the manual mode. (The enable button simply allows developers to observe the state change in the machine model/visualisation in a controlled manner.)

4. SimpleServer and SimpleClient classes

You are already given template files for the SystemJ Controller and a complete model of the Workpiece Loader in `sysj` directory of the Workpiece Loader simulator. As you can see in their configuration files (controller.xml and plant.xml)

<pre>// Program 1 - Controller CD {sustain vacOn;} {sustain armDest;} // Program 2 - Plant CD {await(vacOn);} {await(armDest);}</pre>	<pre><ClockDomain Name="ControllerCD" Class="Controller"> <oSignal Name="vacOn" To="PlantCD.vacOn" Class="com.systemj.ipc.SimpleClient" IP="127.0.0.1" Port="10001" /> <oSignal Name="armDest" To="PlantCD.armDest" Class="com.systemj.ipc.SimpleClient" IP="127.0.0.1" Port="10001" /> </ClockDomain> <ClockDomain Name="PlantCD" Class="Controller"> <iSignal Name="vacOn" Class="com.systemj.ipc.SimpleServer" IP="127.0.0.1" Port="10001" /> <iSignal Name="armDest" Class="com.systemj.ipc.SimpleServer" IP="127.0.0.1" Port="10001" /> </ClockDomain></pre>
(a) SystemJ code	(b) A configuration file



(c) Initialisation of SimpleClient and SimpleServer

Figure 4. Using SimpleServer and SimpleClient for signal communication between two SystemJ programs

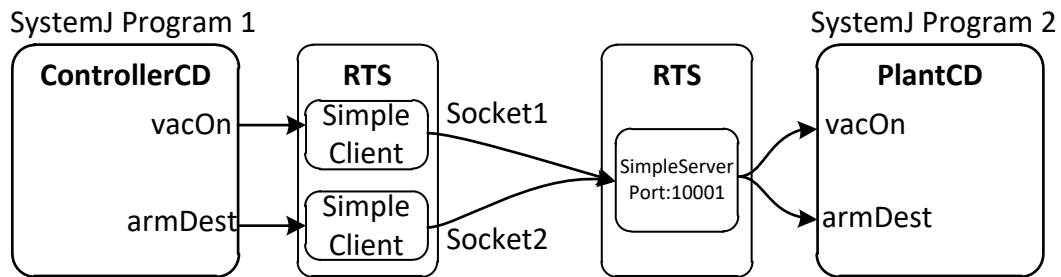


Figure 5. An overview of how runtime accepts incoming connections using SimpleServer and SimpleClient

these two SystemJ programs instantiate objects called `SimpleServer` and `SimpleClient` that are used for input and output signals, respectively. Their implementation is similar to `TCPReceiver` and `TCPSender` introduced in Lab 2 **where a signal is passed as an array of Java objects**. In addition to this, `SimpleClient` first initialises and registers with `SimpleServer` before sending (emitting) any signal objects. For example, consider two SystemJ programs shown in Figure 4(a) where each program runs a single clock-domain, a Controller and a Plant, respectively. In this example, the Controller clock-domain continuously emits signals “vacOn” and “armDest” to the Plant clock-domain and delivery of these signals is handled by the SystemJ runtime system (RTS) using `SimpleServer` and `SimpleClient` classes. A configuration file which declares the clock-domains and their signals is shown in Figure 4(b). When an output signal is emitted by the Controller for the first time, its `SimpleClient` tries to establish and initialise the connection with the `SimpleServer` of the Plant’s runtime system by sending the signal shown as (1) in Figure 4(c). In this case, the client (ControllerCD’s runtime) sends a string “PlantCD.vacOn” (**the value of field “To” in Figure 4(b)**) to the server (PlantCD’s runtime) so that the server knows the client wants to establish a connection with signal vacOn. If there is an input signal called “vacOn” in the PlantCD, the server will reply back with any byte value, otherwise it closes the socket (see (2) in Figure 4(c)). After this initialisation, a signal can be transmitted as an array of objects. For example, see (3) in Figure 4(c) which shows two emissions of signal vacOn with a value of ‘null’ followed by a value of a string ‘Hello’.

When `SimpleServer` is used for input signals, all input signals among multiple clock domains can listen to the same port number for any incoming connections. The SystemJ runtime will then correctly update signal statuses based on the information retrieved during the initialisation step. For example, Figure 5 illustrates how the SystemJ runtime accepts multiple connections using a `SimpleServer`. In this case, two sockets are created, mapped to signals vacOn and armDest, respectively, via a server listening on a port 10001.

In case when a normal Java application (e.g. UI) needs to communicate with a SystemJ program that uses `SimpleServer` and `SimpleClient`, it must perform the initialisation step shown in Figure 5 before exchanging any signal data. For `TCPSender` and `TCPClient`, the initialisation step is **not** needed because connections between input and output signals are already determined via IP address and port number pairs.

5. Exercise – Your Task

5.1 Automatic mode

Develop a SystemJ program that controls the Workpiece Loader device in automatic mode. The initial positions of the components of the Workpiece Loader are:

- Ejecting cylinder (pusher) is retracted
- Rotary arm is extended
- Vacuum is off

The operation sequence of the Workpiece Loader, once a request for a new workpiece arrives, is described as follows:

1. The arm swivels to the position “extended” if a new workpiece is requested (request).
2. The pusher extends and pushes a workpiece out of the magazine.
3. The pusher retracts back once it is fully extended.
4. The arm swivels back to the position “retracted”
5. The vacuum is switched on (wait until the workpiece is securely held, e.g. via WPgripped).

6. The arm swivels to the position “extended”, transferring a workpiece to a pick-up zone.
7. The vacuum is switched off (wait until the workpiece is released, e.g. via !WPgripped).
8. The arm swivels to the position “retracted” so that the cup can be picked up by other machine.

5.2 Manual mode

Extend the SystemJ controller you have developed in Section 5.1 so that the controller can switch its operation mode between ‘automatic’ and ‘manual’ based on the value of the input signal “**mode**” (‘0’ – automatic, and ‘1’ – manual). When the controller is in manual mode, it should be able to generate output signals based on the input signals coming from the UI (e.g. **armDestM**, **armSourceM**, and so on).

Once it is done, show your running simulation to one of the TAs. You will have to develop, in the group project, similar functionalities for multiple intelligent machines and integrate them into the full functional model of a more complex manufacturing system.

References

- [1] H. Park, A. Malik, and Z. Salcic, "Time square - marriage of real-time and logical-time in GALS and synchronous languages," presented at the The 20th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA) Chongqing, China 2014.